

ARCHITECTURE — GateFlow

Versión 2.0 · Evoluciona la arquitectura inicial (Sonnet, sesión previa) hacia una plataforma modular, offline-first, preparada para miles de tenants. Depende de: 00-PRD.md, MISION.md, 01-BUSINESS_RULES.md · Precede a: 03-DATABASE.md, 04-API.md

1. Qué cambia respecto a la arquitectura inicial y por qué

La arquitectura original (Next.js + Supabase + RLS multi-tenant) se mantiene como base — sigue siendo la decisión correcta por las razones ya documentadas: velocidad de ejecución, coherencia con tu experiencia previa, y un modelo de aislamiento por RLS que ya escala al patrón de miles de tenants. Lo que cambia es lo que se construye **sobre** esa base:

1. Se introduce una **capa de sincronización offline-first** real (no un simple “guardar en localStorage y reintentar”) porque el guardia es el usuario primario y su trabajo no puede depender de la señal de la caseta.
2. Se introduce una **capa de módulos** explícita, porque GateFlow deja de ser “una app de paquetes” y pasa a ser una plataforma que absorberá Visitantes, Accesos, Correspondencia, Reservas, Comunicados, Proveedores y Vehículos sin rediseño.
3. Se añaden capas de soporte específicas para las funcionalidades nuevas: ubicación física configurable, generación de código GateFlow, búsqueda universal y dashboard ejecutivo con agregaciones.

Nada de esto invalida el trabajo previo de base de datos — lo extiende. El detalle de esquema se resuelve en `03-DATABASE.md`; este documento define cómo se conectan las piezas.

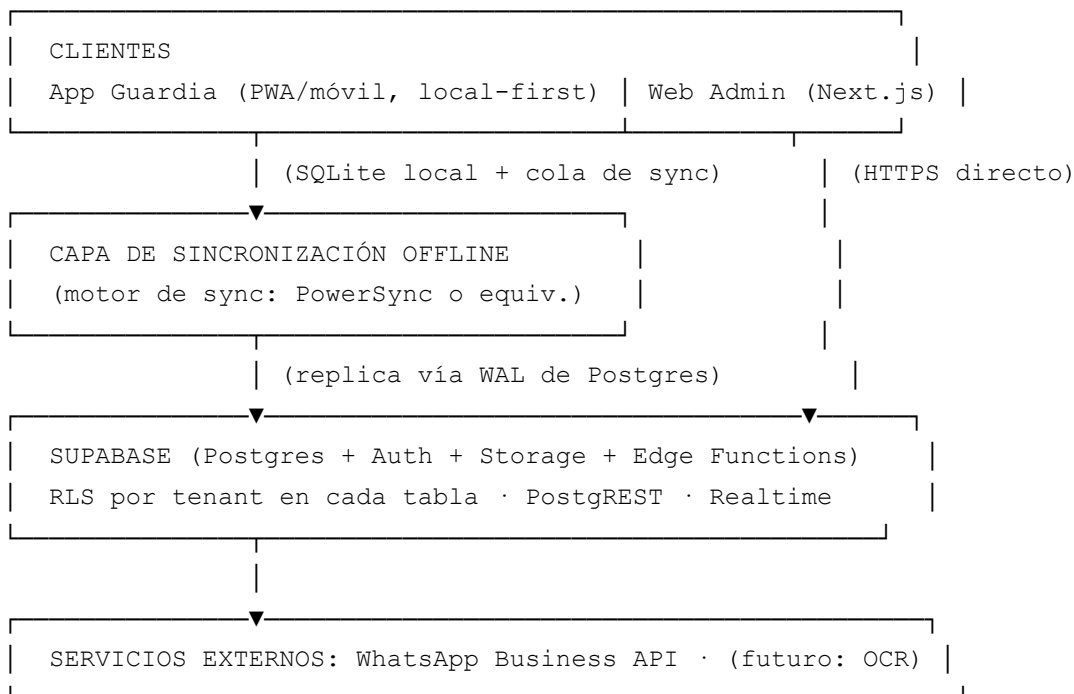
2. Principios arquitectónicos rectores

- **Local-first para el guardia, cloud-first para el administrador.** La experiencia operativa (registro, foto, firma) nunca depende de la red; la experiencia analítica (dashboard, reportes) siempre asume conexión, porque ocurre en un contexto de oficina.
- **Un módulo, un límite claro.** Cada módulo (Paquetes hoy; Visitantes, Accesos, etc. después) es una unidad de código y de dominio independiente que consume las mismas entidades base (tenant, unidad, usuario, rol) pero no depende del código interno de otro

módulo.

- **La base de datos es la autoridad de seguridad, no la aplicación.** Se mantiene y se refuerza: RLS en Postgres, no verificación de tenant en el código de la API.
- **Ningún dato de negocio se destruye.** La arquitectura de almacenamiento asume soft-state e historial inmutable (consistente con BR-13, BR-49, BR-51) — no hay operaciones `DELETE` de negocio en el diseño, solo cambios de estado auditados.
- **Costo proporcional al uso real.** La infraestructura crece por tenant activo y volumen real, no por anticipación — se evita sobre-provisionar cómputo o servicios para una escala que aún no existe.

3. Arquitectura por capas (vista general)



4. Capa de aplicación (frontend)

Se mantiene Next.js + TypeScript para el panel de administración (uso de oficina, con conexión asumida). Para la app de guardia se introduce una distinción arquitectónica importante:

- La app de guardia debe construirse como **PWA local-first** (o app móvil ligera, según se decida en fase de implementación), con una base de datos embebida en el dispositivo (SQLite) que es la que el guardia realmente lee y escribe. La UI nunca espera una

respuesta de red para completar una acción.

- El panel de administración **no** necesita esta capa — su naturaleza es de consulta y configuración, no de captura crítica en campo, y puede depender de conexión sin comprometer la misión del producto.

Esta separación es deliberada: aplicar local-first a todo el sistema por uniformidad sería sobre-ingeniería costosa; aplicarlo solo donde el negocio lo exige (la caseta) es la decisión correcta de simplicidad.

5. Capa de sincronización offline-first

Decisión: usar un motor de sincronización dedicado sobre Postgres/Supabase (PowerSync o equivalente de la misma categoría), en lugar de construir una cola de sincronización propia.

Justificación: construir un motor de sync confiable desde cero (resolución de conflictos, consistencia causal, sincronización parcial por tenant) es un problema de ingeniería no trivial que ya está resuelto por herramientas maduras que se integran nativamente con la Auth y el RLS de Supabase sin requerir cambios de esquema invasivos. Reinventar esto retrasaría el módulo de Paquetes sin aportar ventaja competitiva — la ventaja de GateFlow está en el dominio (portería), no en construir su propio motor de sincronización.

Cómo opera conceptualmente:

1. El dispositivo del guardia mantiene una base de datos local (SQLite) con las tablas relevantes a su tenant.
2. Toda escritura (registrar paquete, subir foto, capturar firma) se guarda primero localmente y queda en una cola de subida.
3. El motor de sincronización reenvía esa cola a Supabase en cuanto hay conectividad, respetando las políticas RLS ya definidas — un guardia solo puede sincronizar datos de su propio tenant, porque las reglas de sincronización usan la misma fuente de autoridad que las políticas de base de datos.
4. Los timestamps de creación se conservan del dispositivo, no del momento de sincronización (consistente con BR-35).
5. En caso de conflicto de escritura sobre el mismo registro (escenario raro dado que cada paquete se crea una sola vez), se aplica la regla de negocio BR-37: gana el registro con timestamp de creación más antiguo; el registro en conflicto se conserva, nunca se descarta.

Las fotografías y firmas se comprimen y guardan localmente primero, subiéndose a

Supabase Storage en cuanto hay red — el registro del paquete no espera a que la foto termine de subir para considerarse completo desde la perspectiva del guardia.

6. Arquitectura modular

Cada módulo (Paquetes ahora; Visitantes, Accesos, Correspondencia, Reservas, Comunicados, Proveedores, Vehículos después) se trata como un dominio independiente que:

- Vive en su propio espacio de tablas, con su propio conjunto de estados y catálogos, pero **siempre referencia** las entidades base compartidas: `tenants`, `unidades`, `users`, `user_tenants`, `roles`, `permisos`.
- No importa ni depende del esquema interno de otro módulo. Un módulo de Visitantes puede *consultar* si una unidad existe, pero nunca debe necesitar conocer las tablas internas de Paquetes para funcionar.
- Expone sus propios permisos dentro del catálogo compartido de `permisos` (ej. `visitantes.crear`, `accesos.autorizar`), reutilizando el mecanismo de roles ya construido, sin crear un sistema de permisos paralelo.
- Puede tener su propia lógica de notificación, pero reutiliza la infraestructura de envío (WhatsApp Business API) ya integrada para Paquetes, en lugar de que cada módulo construya su propio conector.

Esta capa modular es la razón por la que el documento de base de datos (`03-DATABASE.md`) debe distinguir con claridad entre “tablas núcleo de la plataforma” (tenant, usuarios, roles, unidades) y “tablas del módulo Paquetes” — los módulos futuros se apoyan en las primeras y nunca las duplican.

7. Ubicación física de almacenamiento

Se diseña como un **catálogo jerárquico configurable por tenant**, no como campos fijos de “rack/nivel/posición”. Cada residencial define su propia estructura (puede ser tan simple como “Estante A, B, C” o tan detallada como “Rack 3 → Nivel 2 → Posición 05”), y el sistema debe poder representar y mostrar cualquiera de esas estructuras sin necesitar cambios de esquema por tenant. Esto se resuelve con una tabla de ubicaciones auto-referenciada (jerarquía de profundidad variable) en lugar de columnas separadas para cada nivel — el detalle exacto se define en `03-DATABASE.md`, pero la decisión arquitectónica es que la profundidad y nomenclatura de la jerarquía es dato de configuración de cada tenant, no estructura fija de la plataforma.

8. Código interno GateFlow

El código `GF-AAAA-NNNNNNN` se genera de forma que sea:

- **Único globalmente** (no solo por tenant), para que sea seguro imprimirlo, escanearlo por QR y buscarlo sin ambigüedad entre residenciales.
- **Generable sin conexión.** Dado que el guardia puede registrar paquetes offline, la generación del código no puede depender de una consulta al servidor para evitar colisiones — se resuelve con un esquema de generación que combina un identificador de dispositivo/sesión con un contador local y el año, garantizando unicidad al sincronizar (detalle de implementación en `03-DATABASE.md` y `API.md`, sin código en este documento).
- **Representable como QR** de forma trivial, ya que el código es texto plano corto, sin necesidad de un servicio externo de acortamiento o resolución.

9. Búsqueda universal

Se implementa como una **única capa de búsqueda por tenant**, no como consultas separadas por campo. Arquitectónicamente, esto se resuelve con una vista o índice de búsqueda de texto completo (full-text search nativo de Postgres) que combina los campos relevantes (nombre de residente, unidad, teléfono, tracking, código GateFlow, empresa, estado, fecha) en un único vector de búsqueda por registro, filtrado siempre por `tenant_id` vía RLS antes de aplicar el término de búsqueda. Esto evita construir y mantener un motor de búsqueda externo (ej. Elasticsearch) en esta etapa, que sería una complejidad operativa injustificada para el volumen esperado — se reserva como posible evolución futura solo si el volumen de datos por tenant lo exige.

10. Dashboard ejecutivo

Las métricas del dashboard (paquetes de hoy, promedio diario, tiempo promedio hasta entrega, olvidados, top empresas, gráfica de 30 días) no deben calcularse mediante consultas pesadas en tiempo real sobre la tabla transaccional de paquetes en cada carga del dashboard — eso no escala cuando un tenant acumula años de historial. Se resuelve con **vistas materializadas o tablas de agregación pre-calculadas**, actualizadas periódicamente (ej. cada pocos minutos vía Edge Function programada), separando la escritura transaccional (rápida, un paquete a la vez) de la lectura analítica (agregada, tolerante a algunos minutos de desactualización). Esta separación es estándar y evita que el crecimiento de datos de un tenant afecte el rendimiento de otro.

11. Preparación para OCR/IA futura

No se implementa en esta fase, pero la arquitectura no debe impedirlo. Esto se traduce en dos decisiones concretas:

- El flujo de registro de paquete debe permitir, en el futuro, que la fotografía capturada se envíe opcionalmente a un servicio de extracción de datos (vía Edge Function) que devuelva campos sugeridos (nombre, dirección, tracking, courier) para pre-llenar el formulario — esto requiere que la captura de foto ocurra *antes* de solicitar esos datos al guardia, no después, lo cual ya es consistente con el flujo UX definido.
- El modelo de datos debe guardar la foto original de forma que pueda reprocesarse retroactivamente si el servicio de OCR mejora o se incorpora después — es decir, las fotografías no son “descartables” una vez extraída la información, se conservan (consistente con BR-28).

12. Seguridad

Se mantiene y se refuerza el modelo original:

- RLS en Postgres como autoridad única de aislamiento entre tenants (BR-01 a BR-04).
- JWT con `tenant_id` y `role` inyectados vía Auth Hook, ahora también consumidos por la capa de sincronización offline para que las reglas de sync respeten el mismo aislamiento que las políticas de base de datos.
- Los datos sincronizados a un dispositivo offline deben limitarse estrictamente a los del tenant activo del guardia — un dispositivo nunca debe almacenar localmente datos de un residencial al que el guardia no pertenece actualmente.
- El almacenamiento local en el dispositivo (SQLite embebido) debe considerarse dato sensible: si el dispositivo se pierde o es robado, la exposición se limita a un solo tenant, nunca a la plataforma completa — otra razón para no centralizar todo el catálogo de tenants en cada dispositivo.

13. Escalabilidad a miles de tenants

Sin cambios de fondo respecto al documento de arquitectura original: shared database + RLS, compute escalable por tier en Supabase, particionamiento futuro de tablas de alto volumen (`paquetes` , `audit_log`) por rango de fecha cuando el volumen lo justifique. Lo que se añade en esta revisión:

- Las vistas materializadas del dashboard (sección 10) deben particionarse

conceptualmente por tenant desde el diseño, para que el crecimiento de un residencial grande no degrade el tiempo de agregación de los demás.

- La capa de sincronización offline debe soportar sync selectivo por tenant (no sincroniza “toda la base”, solo el subconjunto del tenant y unidad del dispositivo), lo cual es una capacidad nativa de las herramientas de sync mencionadas en la sección 5, no algo que deba construirse a medida.

14. Qué NO cambia (para evitar interpretaciones erróneas)

- Supabase sigue siendo el backend primario. No se migra a una arquitectura de microservicios.
- El modelo multi-tenant sigue siendo shared-schema + RLS, no schema-per-tenant ni database-per-tenant.
- Netlify/Vercel siguen siendo la plataforma de deployment del frontend administrativo.
- No se introduce Elasticsearch, Kafka, ni infraestructura de streaming — serían complejidad operativa desproporcionada al volumen actual y esperado en el corto/mediano plazo.